Soren Thumann

EECS 348

29 October 2026

Assignment 5 GenAI Analysis

For this assignment, the two generative AI programs used in this assignment were ChatGPT.com and Claude.ai. They were accessed through a Google query.

Identical prompts were given to both programs. The prompt read:

Edit this recursive object-oriented C++ program provided in the file that can solve a partially complete Sudoku puzzle. The program must be edited such that the output information of each puzzle is written to a file named solution.txt, where N is the puzzle number. The number of solution files should match the number of input files. Clearly mark where edits were made, or new code was written. The following parameters are previous requirements for the code that must be maintained: This code must use objects, not functions. The partially complete Sudoku puzzle will be provided through an input file. The puzzles are 9x9 grids with 3x3 sub grids. Blank spaces are indicated with underscores. Standard rules of Sudoku apply. The goal of the program is to find the solution (or at least a solution) to the puzzle. If no solution exists, this should be proven. If there is more than one solution, all solutions should be printed out. The output for the program should include the input puzzle file name (e.g., puzzle1.txt), the puzzle from the file printed out, and the solution(s) to the puzzle printed out or if no solution is found, "No solution found" printed out. The puzzle should be solved using the depth-first search with backtracking. Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).
Example Input file:
5 _ _ _ _ _ 1 7 _

1 _ 6 5 _ 9 _ 4 _
4 7 2 1 _ 6 _ _ _
9 _ _ _ _ _ 5 _ _
_ 1 8 _ 9 5 4 _ _
6 _ _ 4 _ 2 3 8 9
_ 4 _ _ _ _ 9 3 _
_ 9 _ 7 _ 3 _ 5 _
2 6 3 9 5 8 7 1 4
Example Output file:
5 3 9 8 2 4 1 7 6
1 8 6 5 7 9 2 4 3
4 7 2 1 3 6 8 9 5
9 2 4 3 8 7 5 6 1
3 1 8 6 9 5 4 2 7
6 5 7 4 1 2 3 8 9
7 4 5 2 6 1 9 3 8
8 9 1 7 4 3 6 5 2
2 6 3 9 5 8 7 1 4
Solutions found: 1

ChatGPT Generated Code:

```
/* EECS 348 Assignment 4

 *  This program is a sodoku solver meant to brute-force solutions for the timeless

 *  puzzle type.

 *  Collaborators: Luke Tidball, ChatGPT 6

 */



#include <iostream> // Provides console input and output.

#include <fstream> // Provides file input and output.

#include <string> // Provides the string type and to_string function.

#include <vector> // [EDIT] Stores the discovered input puzzle filenames.

#include <algorithm> // [EDIT] Sorts puzzle filenames by puzzle number.



using namespace std; // Allows standard library names to be used without std::.



class Sudoku { // Defines a class for loading, displaying, and solving Sudoku puzzles.
```

```
private: // Begins the section accessible only inside the class.
    static const int SIZE = 9; // Sets the number of rows and columns in the grid.
    int grid[SIZE][SIZE]; // Stores the Sudoku cells, with zero representing an empty cell.


    // Checks whether a number can be placed in a cell.
    bool isValid(int row, int col, int num) const { // Returns whether num can go at the specified row and column.
        // Check the row.
        for (int c = 0; c < SIZE; c++) { // Examines every column in the specified row.
            if (grid[row][c] == num) { // Checks whether the number already appears in this row.
                return false; // Rejects the number because it would duplicate a row value.
            } // Ends the row-conflict check.
        } // Ends the loop that checks the row.


        // Check the column.
        for (int r = 0; r < SIZE; r++) { // Examines every row in the specified column.
```

```
        if (grid[r][col] == num) { // Checks whether the number already appears in this column.
            return false; // Rejects the number because it would duplicate a column value.
        } // Ends the column-conflict check.
    } // Ends the loop that checks the column.


    // Find the top-left corner of the 3x3 block.
    int blockRow = (row / 3) * 3; // Calculates the first row of the cell's 3x3 block.
    int blockCol = (col / 3) * 3; // Calculates the first column of the cell's 3x3 block.


    // Check the 3x3 block.
    for (int r = blockRow; r < blockRow + 3; r++) { // Visits each row in the cell's 3x3 block.
        for (int c = blockCol; c < blockCol + 3; c++) { // Visits each column in the current block row.
            if (grid[r][c] == num) { // Checks whether the number already appears in this block.
                return false; // Rejects the number because it would duplicate a block value.
            } // Ends the block-conflict check.
```

```
            } // Ends the loop through the block's columns.

        } // Ends the loop through the block's rows.



        return true; // Allows the number because no row, column, or block conflict was found.

    } // Ends the isValid function.



    // Counts how many valid numbers can be placed in an empty cell.

    int countCandidates(int row, int col) const { // Returns the number of valid digits for the specified cell.

        int count = 0; // Stores the number of valid candidates found.



        // Try every possible Sudoku digit, from 1 through 9.

        for (int num = 1; num <= 9; num++) { // Tests each possible Sudoku digit.



            // Check whether the current digit follows Sudoku's rules.

            if (isValid(row, col, num)) { // Checks whether num can be placed in the specified cell.
```

```
            count++; // Increase the count if the digit is valid.
        } // Ends the check for the current candidate.
    } // Ends the loop through possible digits.

    return count; // Return the total number of valid candidates.
} // Ends the countCandidates function.

// Finds the empty cell with the fewest valid candidate numbers.
bool findBestEmptyCell(int &bestRow, int &bestCol) const { // Finds an empty cell with the fewest candidates and saves its coordinates.
    int minCandidates = 10; // Start above the maximum of nine candidates.
    bool foundEmpty = false; // Tracks whether an empty cell exists.
    // Examine each row of the Sudoku grid.
    for (int row = 0; row < SIZE; row++) { // Visits every row in the grid.
        // Examine each column in the current row.
```

```
        for (int col = 0; col < SIZE; col++) { // Visits every column in the current row.
            // Check whether the current cell is empty.
            if (grid[row][col] == 0) { // Checks whether the value at grid[row][col] is zero.
                foundEmpty = true; // Record that an empty cell exists.
                // Count the valid numbers for this empty cell.
                int candidates = countCandidates(row, col); // Counts valid digits for this empty cell.
                // A cell with zero candidates cannot be solved.
                if (candidates == 0) { // Checks whether this empty cell has no valid candidates.
                    bestRow = row; // Save the impossible cell's row.
                    bestCol = col; // Save the impossible cell's column.
                    return true; // Returns so the solver can detect failure.
                } // Ends the check for a cell with no candidates.
                // Check whether this cell has fewer candidates than before.
                if (candidates < minCandidates) { // Checks whether this cell has fewer candidates than the current best.
                    minCandidates = candidates; // Update the smallest count.
```

```
                    bestRow = row; // Save the selected cell's row.

                    bestCol = col; // Save the selected cell's column.

                } // Ends the check for a new best cell.

                // Stop searching if a cell has only one candidate.

                if (minCandidates == 1) { // Checks whether the smallest candidate count is one.

                    return true; // Returns because one is the smallest positive candidate count.

                } // Ends the check for a single-candidate cell.

            } // Ends the check for an empty cell.

        } // Ends the loop through columns.

    } // Ends the loop through rows.



    // Return true if an empty cell was found, or false if the grid is full.

    return foundEmpty; // Reports whether an empty cell was found.

} // Ends the findBestEmptyCell function.



// Recursively solves the puzzle using depth-first search and backtracking.
```

```
long long solveRecursive(ostream &output) { // [EDIT] Writes every solution to the selected output file.

    int row, col; // Store the coordinates of the selected empty cell.

    // Find the empty cell with the fewest valid candidates.
    if (!findBestEmptyCell(row, col)) { // Checks whether there are no empty cells left.

        printGrid(output); // [EDIT] Writes the completed Sudoku solution to the output file.

        return 1; // Reports that one valid solution has been found.
    } // Ends the check for a completed grid.

    // Check whether the selected cell has any valid candidates.
    if (countCandidates(row, col) == 0) { // Checks whether the selected cell has no valid digits.
```

```
    return 0; // Reports that this search path cannot produce a solution.

} // Ends the check for a cell with no candidates.



long long solutions = 0; // Counts the solutions found in this search.



// Try every possible digit from 1 through 9.

for (int num = 1; num <= 9; num++) { // Tests each possible Sudoku digit.



    // Only attempt to place a digit if it follows Sudoku's rules.

    if (isValid(row, col, num)) { // Checks whether num can be placed in the selected cell.



        grid[row][col] = num; // Places the candidate in the selected cell.



        // Recursively search for solutions with this digit in place.

        solutions += solveRecursive(output); // [EDIT] Recurses while keeping all output in the same file.
```

```
            // Remove the digit so another candidate can be tested.
            grid[row][col] = 0; // Clears the cell to try another candidate.
        } // Ends the check for the current digit.
    } // Ends the loop through possible digits.

    return solutions; // Returns the total solutions found in this search.
} // Ends the solveRecursive function.

    // Checks whether the original puzzle has conflicting clues.
    bool hasValidStartingGrid() { // Returns whether the starting clues follow Sudoku rules.
        for (int row = 0; row < SIZE; row++) { // Visits every row in the grid.
            for (int col = 0; col < SIZE; col++) { // Visits every column in the current row.
                int num = grid[row][col]; // Stores the current cell's value.

                if (num == 0) { // Checks whether the current cell is empty.
```

```
            continue; // Skips empty cells because they are not starting clues.

        } // Ends the empty-cell check.



        // Temporarily remove the clue to check for conflicts.

        grid[row][col] = 0; // Clears the clue so isValid can check the other cells.



        bool valid = isValid(row, col, num); // Checks whether the clue conflicts with the rest of the grid.



        // Restore the original clue.

        grid[row][col] = num; // Puts the original clue back in its cell.



        if (!valid) { // Checks whether the clue conflicts with another value.

            return false; // Reports that the starting grid is invalid.

        } // Ends the conflict check.

    } // Ends the loop through columns.
```

```
        } // Ends the loop through rows.



        return true; // Reports that no conflicting starting clues were found.

    } // Ends the hasValidStartingGrid function.



public: // Begins the section accessible to code that uses the class.

    // Initializes an empty Sudoku grid.

    Sudoku() { // Defines the constructor that initializes a Sudoku object.

        for (int row = 0; row < SIZE; row++) { // Visits every row in the grid.

            for (int col = 0; col < SIZE; col++) { // Visits every column in the current row.

                grid[row][col] = 0; // Marks the current cell as empty.

            } // Ends the loop through columns.

        } // Ends the loop through rows.

    } // Ends the Sudoku constructor.



    // Reads a Sudoku puzzle from a file.
```

```
    bool loadPuzzle(const string &filename) { // Loads a puzzle file and reports whether it was read successfully.

        ifstream inputFile(filename); // Opens the specified file for input.



        if (!inputFile.is_open()) { // Checks whether the file failed to open.

            cout << "Error: Could not open " << filename << endl; // Displays an error message with the filename.

            return false; // Reports that the puzzle could not be loaded.

        } // Ends the file-open check.



        for (int row = 0; row < SIZE; row++) { // Visits each row of cells to read from the file.

            for (int col = 0; col < SIZE; col++) { // Visits each column of the current row.

                string cell; // Stores the next cell value read from the file.



                if (!(inputFile >> cell)) { // Checks whether a required cell could not be read.

                    cout << "Error: Invalid puzzle format in " // Begins displaying an invalid-format error.
```

```
                << filename << endl; // Adds the filename and ends the error message.

            inputFile.close(); // Closes the file after the read error.

            return false; // Reports that the puzzle could not be loaded.

        } // Ends the check for a missing cell.



        if (cell == "_") { // Checks whether the file marks this cell as empty.

            grid[row][col] = 0; // Stores zero to represent an empty cell.

        } else if (cell.length() == 1 && // Checks that the cell contains exactly one character.

                cell[0] >= '1' && cell[0] <= '9') { // Checks that the character is a digit from 1 through 9.

            grid[row][col] = cell[0] - '0'; // Converts the digit character to an integer and stores it.

        } else { // Handles any cell value that is neither an underscore nor a valid digit.

            cout << "Error: Invalid cell in " // Begins displaying an invalid-cell error.

                << filename << endl; // Adds the filename and ends the error message.

            inputFile.close(); // Closes the file after the invalid cell.
```

```
            return false; // Reports that the puzzle could not be loaded.

        } // Ends the cell-value checks.

    } // Ends the loop through columns.

} // Ends the loop through rows.



// Ensure the file contains exactly 81 cells.

string extra; // Stores a possible value remaining after the expected cells are read.



if (inputFile >> extra) { // Checks whether the file contains another value after the 81 cells.

    cout << "Error: Too many cells in " // Begins displaying a too-many-cells error.

         << filename << endl; // Adds the filename and ends the error message.

    inputFile.close(); // Closes the file after detecting extra content.

    return false; // Reports that the puzzle file has an invalid format.

} // Ends the check for extra cells.



inputFile.close(); // Closes the file after reading the puzzle.
```

```
        return true; // Reports that the puzzle was loaded successfully.

    } // Ends the loadPuzzle function.



    // Prints the Sudoku grid.

    void printGrid(ostream &output) const { // [EDIT] Displays the grid in the supplied output stream.

        for (int row = 0; row < SIZE; row++) { // Visits every row in the grid.

            if (row % 3 == 0) { // Checks whether this row begins a 3-row block.

                output << "+-------+-------+-------+" << endl; // Prints a horizontal separator between blocks.

            } // Ends the block-row check.



            for (int col = 0; col < SIZE; col++) { // Visits every column in the current row.

                if (col % 3 == 0) { // Checks whether this column begins a 3-column block.

                    output << "| "; // Prints a vertical separator before the block.

                } // Ends the block-column check.
```

```
            if (grid[row][col] == 0) { // Checks whether the current cell is empty.
                output << "_ "; // Prints an underscore for an empty cell.
            } else { // Handles a cell that contains a digit.
                output << grid[row][col] << ' '; // Prints the digit followed by a space.
            } // Ends the empty-cell check.
        } // Ends the loop through columns.

        output << "|" << endl; // Prints the right edge of the row and moves to the next line.
    } // Ends the loop through rows.

    output << "+-------+-------+-------+" << endl; // Prints the bottom border of the grid.
} // Ends the printGrid function.

// Solves the puzzle and prints all solutions.
```

```
    void solve(ostream &output) { // [EDIT] Solves the puzzle and writes the result to the output file.

        if (!hasValidStartingGrid()) { // Checks whether the starting clues conflict.

            output << "No solution found" << endl; // Reports that the invalid starting grid has no solution.

            return; // Stops solving this puzzle.

        } // Ends the starting-grid validation check.



        long long solutions = solveRecursive(output); // Searches for solutions and stores the number found.



        if (solutions == 0) { // Checks whether the search found no solutions.

            output << "No solution found" << endl; // Reports that no valid solution exists.

        } else { // Handles the case where one or more solutions were found.

            output << "Total solutions found: " // Begins displaying the number of solutions.

                << solutions << endl; // Prints the solution count and ends the line.

        } // Ends the check for whether solutions were found.
```

```
    } // Ends the solve function.

}; // Ends the Sudoku class definition.



int main() { // Begins the program's main function.

    // [EDIT] Discover sequentially numbered puzzle files and create one matching

    // solution file for each input file that actually exists.

    int puzzleNumber = 1;

    int puzzlesProcessed = 0;



    while (true) {

        string filename = "../puzzle" + to_string(puzzleNumber) + ".txt";

        ifstream inputCheck(filename);



        // Stop when the next sequential input file does not exist.

        if (!inputCheck.is_open()) {

            break;
```

```
}

inputCheck.close();



// [EDIT] Name the output file solutionN.txt to match puzzleN.txt.

string outputFilename = "../solution" + to_string(puzzleNumber) + ".txt";

ofstream outputFile(outputFilename);



if (!outputFile.is_open()) {

    cerr << "Error: Could not create " << outputFilename << endl;

    return 1;

}



outputFile << "====================================" << endl;

outputFile << "Puzzle file: puzzle" << puzzleNumber << ".txt" << endl;

outputFile << "====================================" << endl;
```

```
Sudoku sudoku; // Creates a Sudoku object with an initially empty grid.



if (sudoku.loadPuzzle(filename)) {

    outputFile << "\nOriginal puzzle:" << endl;

    sudoku.printGrid(outputFile); // [EDIT] Original puzzle goes to solutionN.txt.



    outputFile << "\nSolution(s):" << endl;

    sudoku.solve(outputFile); // [EDIT] All solutions/count or no-solution result go to file.

} else {

    // [EDIT] Record input errors in the corresponding output file.

    outputFile << "Could not load puzzle." << endl;

}



outputFile.close();

++puzzlesProcessed;

++puzzleNumber;
```

```
    }



    // [EDIT] Console output is only a brief completion message; puzzle details

    // are written exclusively to the corresponding solutionN.txt files.

    cout << "Processed " << puzzlesProcessed

        << " puzzle file(s). Output written to solutionN.txt file(s)." << endl;



    return 0;

} // Ends the main function.
```

Claude.ai Generated Code:

```
/* EECS 348 Assignment 4
 *  This program is a sodoku solver meant to brute-force solutions for the timeless
 *  puzzle type.
 *  Collaborators: Luke Tidball, ChatGPT 6
 */

#include <iostream> // Provides console input and output.
#include <fstream> // Provides file input and output.
#include <string> // Provides the string type and to_string function.
#include <ostream> // EDIT: Provides ostream so output can be directed to a file.

using namespace std; // Allows standard library names to be used without std::.

class Sudoku { // Defines a class for loading, displaying, and solving Sudoku puzzles.
```

```
private: // Begins the section accessible only inside the class.
    static const int SIZE = 9; // Sets the number of rows and columns in the grid.
    int grid[SIZE][SIZE]; // Stores the Sudoku cells, with zero representing an empty cell.


    // Checks whether a number can be placed in a cell.
    bool isValid(int row, int col, int num) const { // Returns whether num can go at the specified row and column.
        // Check the row.
        for (int c = 0; c < SIZE; c++) { // Examines every column in the specified row.
            if (grid[row][c] == num) { // Checks whether the number already appears in this row.
                return false; // Rejects the number because it would duplicate a row value.
            } // Ends the row-conflict check.
        } // Ends the loop that checks the row.


        // Check the column.
        for (int r = 0; r < SIZE; r++) { // Examines every row in the specified column.
```

```
        if (grid[r][col] == num) { // Checks whether the number already appears in this column.
            return false; // Rejects the number because it would duplicate a column value.
        } // Ends the column-conflict check.
    } // Ends the loop that checks the column.


    // Find the top-left corner of the 3x3 block.
    int blockRow = (row / 3) * 3; // Calculates the first row of the cell's 3x3 block.
    int blockCol = (col / 3) * 3; // Calculates the first column of the cell's 3x3 block.


    // Check the 3x3 block.
    for (int r = blockRow; r < blockRow + 3; r++) { // Visits each row in the cell's 3x3 block.
        for (int c = blockCol; c < blockCol + 3; c++) { // Visits each column in the current block row.
            if (grid[r][c] == num) { // Checks whether the number already appears in this block.
                return false; // Rejects the number because it would duplicate a block value.
            } // Ends the block-conflict check.
```

```
            } // Ends the loop through the block's columns.

        } // Ends the loop through the block's rows.



        return true; // Allows the number because no row, column, or block conflict was found.

    } // Ends the isValid function.



    // Counts how many valid numbers can be placed in an empty cell.

    int countCandidates(int row, int col) const { // Returns the number of valid digits for the specified cell.

        int count = 0; // Stores the number of valid candidates found.



        // Try every possible Sudoku digit, from 1 through 9.

        for (int num = 1; num <= 9; num++) { // Tests each possible Sudoku digit.



            // Check whether the current digit follows Sudoku's rules.

            if (isValid(row, col, num)) { // Checks whether num can be placed in the specified cell.
```

```
            count++; // Increase the count if the digit is valid.
        } // Ends the check for the current candidate.
    } // Ends the loop through possible digits.

    return count; // Return the total number of valid candidates.
} // Ends the countCandidates function.

// Finds the empty cell with the fewest valid candidate numbers.
bool findBestEmptyCell(int &bestRow, int &bestCol) const { // Finds an empty cell with the fewest candidates and saves its coordinates.
    int minCandidates = 10; // Start above the maximum of nine candidates.
    bool foundEmpty = false; // Tracks whether an empty cell exists.
    // Examine each row of the Sudoku grid.
    for (int row = 0; row < SIZE; row++) { // Visits every row in the grid.
        // Examine each column in the current row.
```

```
        for (int col = 0; col < SIZE; col++) { // Visits every column in the current row.
            // Check whether the current cell is empty.
            if (grid[row][col] == 0) { // Checks whether the value at grid[row][col] is zero.
                foundEmpty = true; // Record that an empty cell exists.
                // Count the valid numbers for this empty cell.
                int candidates = countCandidates(row, col); // Counts valid digits for this empty cell.
                // A cell with zero candidates cannot be solved.
                if (candidates == 0) { // Checks whether this empty cell has no valid candidates.
                    bestRow = row; // Save the impossible cell's row.
                    bestCol = col; // Save the impossible cell's column.
                    return true; // Returns so the solver can detect failure.
                } // Ends the check for a cell with no candidates.
                // Check whether this cell has fewer candidates than before.
                if (candidates < minCandidates) { // Checks whether this cell has fewer candidates than the current best.
                    minCandidates = candidates; // Update the smallest count.
```

```
                    bestRow = row; // Save the selected cell's row.

                    bestCol = col; // Save the selected cell's column.

                } // Ends the check for a new best cell.

                // Stop searching if a cell has only one candidate.

                if (minCandidates == 1) { // Checks whether the smallest candidate count is one.

                    return true; // Returns because one is the smallest positive candidate count.

                } // Ends the check for a single-candidate cell.

            } // Ends the check for an empty cell.

        } // Ends the loop through columns.

    } // Ends the loop through rows.



    // Return true if an empty cell was found, or false if the grid is full.

    return foundEmpty; // Reports whether an empty cell was found.

} // Ends the findBestEmptyCell function.



// Recursively solves the puzzle using depth-first search and backtracking.
```

long long solveRecursive(ostream &out) { // EDIT: now takes the output stream (the solution file). Searches for solutions and returns how many it finds.

int row, col; // Store the coordinates of the selected empty cell.

// Find the empty cell with the fewest valid candidates.

if (!findBestEmptyCell(row, col)) { // Checks whether there are no empty cells left.

printGrid(out); // EDIT: Prints the completed Sudoku solution to the solution file.

return 1; // Reports that one valid solution has been found.

} // Ends the check for a completed grid.

// Check whether the selected cell has any valid candidates.

if (countCandidates(row, col) == 0) { // Checks whether the selected cell has no valid digits.

```
        return 0; // Reports that this search path cannot produce a solution.

    } // Ends the check for a cell with no candidates.



    long long solutions = 0; // Counts the solutions found in this search.



    // Try every possible digit from 1 through 9.

    for (int num = 1; num <= 9; num++) { // Tests each possible Sudoku digit.



        // Only attempt to place a digit if it follows Sudoku's rules.

        if (isValid(row, col, num)) { // Checks whether num can be placed in the selected cell.



            grid[row][col] = num; // Places the candidate in the selected cell.



            // Recursively search for solutions with this digit in place.

            solutions += solveRecursive(out); // EDIT: passes the output stream along. Searches the updated grid and adds its solution count.
```

```
            // Remove the digit so another candidate can be tested.
            grid[row][col] = 0; // Clears the cell to try another candidate.
        } // Ends the check for the current digit.
    } // Ends the loop through possible digits.

    return solutions; // Returns the total solutions found in this search.
} // Ends the solveRecursive function.

    // Checks whether the original puzzle has conflicting clues.
    bool hasValidStartingGrid() { // Returns whether the starting clues follow Sudoku rules.
        for (int row = 0; row < SIZE; row++) { // Visits every row in the grid.
            for (int col = 0; col < SIZE; col++) { // Visits every column in the current row.
                int num = grid[row][col]; // Stores the current cell's value.

                if (num == 0) { // Checks whether the current cell is empty.
```

```
            continue; // Skips empty cells because they are not starting clues.

        } // Ends the empty-cell check.



        // Temporarily remove the clue to check for conflicts.

        grid[row][col] = 0; // Clears the clue so isValid can check the other cells.



        bool valid = isValid(row, col, num); // Checks whether the clue conflicts with the rest of the grid.



        // Restore the original clue.

        grid[row][col] = num; // Puts the original clue back in its cell.



        if (!valid) { // Checks whether the clue conflicts with another value.

            return false; // Reports that the starting grid is invalid.

        } // Ends the conflict check.

    } // Ends the loop through columns.
```

```
        } // Ends the loop through rows.



        return true; // Reports that no conflicting starting clues were found.

    } // Ends the hasValidStartingGrid function.



public: // Begins the section accessible to code that uses the class.

    // Initializes an empty Sudoku grid.

    Sudoku() { // Defines the constructor that initializes a Sudoku object.

        for (int row = 0; row < SIZE; row++) { // Visits every row in the grid.

            for (int col = 0; col < SIZE; col++) { // Visits every column in the current row.

                grid[row][col] = 0; // Marks the current cell as empty.

            } // Ends the loop through columns.

        } // Ends the loop through rows.

    } // Ends the Sudoku constructor.



    // Reads a Sudoku puzzle from a file.
```

```
    bool loadPuzzle(const string &filename, ostream &out) { // EDIT: error messages now go to the solution file (out).

    // // Loads a puzzle file and reports whether it was read successfully.

        ifstream inputFile(filename); // Opens the specified file for input.



        if (!inputFile.is_open()) { // Checks whether the file failed to open.

            out << "Error: Could not open " << filename << endl; // EDIT: written to file instead of cout.

            // // Displays an error message with the filename.

            return false; // Reports that the puzzle could not be loaded.

        } // Ends the file-open check.



        for (int row = 0; row < SIZE; row++) { // Visits each row of cells to read from the file.

            for (int col = 0; col < SIZE; col++) { // Visits each column of the current row.

                string cell; // Stores the next cell value read from the file.
```

```
            if (!(inputFile >> cell)) { // Checks whether a required cell could not be read.

                out << "Error: Invalid puzzle format in " // EDIT: written to file instead of cout.
Begins displaying an invalid-format error.

                    << filename << endl; // Adds the filename and ends the error message.

                inputFile.close(); // Closes the file after the read error.

                return false; // Reports that the puzzle could not be loaded.

            } // Ends the check for a missing cell.



            if (cell == "_") { // Checks whether the file marks this cell as empty.

                grid[row][col] = 0; // Stores zero to represent an empty cell.

            } else if (cell.length() == 1 && // Checks that the cell contains exactly one character.

                    cell[0] >= '1' && cell[0] <= '9') { // Checks that the character is a digit from 1
through 9.

                grid[row][col] = cell[0] - '0'; // Converts the digit character to an integer and stores
it.

            } else { // Handles any cell value that is neither an underscore nor a valid digit.
```

```
                out << "Error: Invalid cell in " // EDIT: written to file instead of cout. Begins displaying an invalid-cell error.
                    << filename << endl; // Adds the filename and ends the error message.
                inputFile.close(); // Closes the file after the invalid cell.
                return false; // Reports that the puzzle could not be loaded.
            } // Ends the cell-value checks.
        } // Ends the loop through columns.
    } // Ends the loop through rows.

    // Ensure the file contains exactly 81 cells.
    string extra; // Stores a possible value remaining after the expected cells are read.

    if (inputFile >> extra) { // Checks whether the file contains another value after the 81 cells.
        out << "Error: Too many cells in " // EDIT: written to file instead of cout. Begins displaying a too-many-cells error.
            << filename << endl; // Adds the filename and ends the error message.
```

```
            inputFile.close(); // Closes the file after detecting extra content.

            return false; // Reports that the puzzle file has an invalid format.

        } // Ends the check for extra cells.



        inputFile.close(); // Closes the file after reading the puzzle.

        return true; // Reports that the puzzle was loaded successfully.

    } // Ends the loadPuzzle function.



    // Prints the Sudoku grid.

    // EDIT: every cout in this function was changed to out (the solution file).

    void printGrid(ostream &out) const { // EDIT: takes an output stream so it can print to a file.

    // // Displays the current Sudoku grid in the console.

        for (int row = 0; row < SIZE; row++) { // Visits every row in the grid.

            if (row % 3 == 0) { // Checks whether this row begins a 3-row block.

                out << "+-------+-------+-------+" << endl; // Prints a horizontal separator between
blocks.
```

```
    } // Ends the block-row check.



    for (int col = 0; col < SIZE; col++) { // Visits every column in the current row.

        if (col % 3 == 0) { // Checks whether this column begins a 3-column block.

            out << "| "; // Prints a vertical separator before the block.

        } // Ends the block-column check.



        if (grid[row][col] == 0) { // Checks whether the current cell is empty.

            out << "_ "; // Prints an underscore for an empty cell.

        } else { // Handles a cell that contains a digit.

            out << grid[row][col] << ' '; // Prints the digit followed by a space.

        } // Ends the empty-cell check.

    } // Ends the loop through columns.



    out << "|" << endl; // Prints the right edge of the row and moves to the next line.

} // Ends the loop through rows.
```

```
        out << "+-------+-------+-------+" << endl; // Prints the bottom border of the grid.

    } // Ends the printGrid function.



    // Solves the puzzle and prints all solutions.

    // EDIT: every cout in this function was changed to out (the solution file).

    void solve(ostream &out) { // EDIT: takes the solution-file stream.

    // // Checks the starting grid, searches for solutions, and reports the result.

        if (!hasValidStartingGrid()) { // Checks whether the starting clues conflict.

            out << "No solution found" << endl; // Reports that the invalid starting grid has no solution.

            return; // Stops solving this puzzle.

        } // Ends the starting-grid validation check.



        long long solutions = solveRecursive(out); // Searches for solutions and stores the number found.
```

```
        if (solutions == 0) { // Checks whether the search found no solutions.
            out << "No solution found" << endl; // Reports that no valid solution exists.
        } else { // Handles the case where one or more solutions were found.
            out << "Total solutions found: " // Begins displaying the number of solutions.
                << solutions << endl; // Prints the solution count and ends the line.
        } // Ends the check for whether solutions were found.
    } // Ends the solve function.
}; // Ends the Sudoku class definition.


int main() { // Begins the program's main function.
    // Process all five puzzle files.
    for (int i = 1; i <= 5; i++) { // Loops through puzzle numbers 1 to 5.
        string filename = "../puzzle" + to_string(i) + ".txt"; // Builds the path for the current puzzle file.
```

```
// ===== EDIT START: skip puzzle numbers with no input file so solution files match input files =====

ifstream testFile(filename); // Tries to open the input file just to see if it exists.

if (!testFile.is_open()) { // Checks whether this input file is missing.

    cout << "Could not open " << filename << " - no solution file created." << endl; // Notes the skip on the console.

    continue; // Moves on without creating a solution file.

} // Ends the existence check.

testFile.close(); // Closes the test stream.



string outName = "../solution" + to_string(i) + ".txt"; // Builds the output file name solutionN.txt.

ofstream outFile(outName); // Opens the solution file for puzzle N.

if (!outFile.is_open()) { // Checks whether the solution file could not be created.

    cout << "Error: Could not create " << outName << endl; // Reports the failure on the console.

    continue; // Skips this puzzle.
```

} // Ends the output-open check.

outFile << "====================================" << endl; // Prints a separator at the top of the solution file.

outFile << "Puzzle file: " << filename << endl; // Writes the input puzzle file name to the solution file.

outFile << "====================================" << endl; // Prints a separator below the file name.

Sudoku sudoku; // Creates a Sudoku object with an initially empty grid.

if (sudoku.loadPuzzle(filename, outFile)) { // Loads the puzzle; load errors go to the solution file.

outFile << "\nOriginal puzzle:" << endl; // Writes a label for the puzzle as loaded.

sudoku.printGrid(outFile); // Writes the original puzzle grid to the solution file.

outFile << "\nSolution(s):" << endl; // Writes a label before the solver's output.

```
            sudoku.solve(outFile); // Solves the puzzle, writing all solutions (or "No solution found") to the file.

        } // Ends the check for successful puzzle loading.



        outFile.close(); // Closes the solution file.

        cout << "Wrote " << outName << " for " << filename << endl; // Tells the user on the console which file was written.

        // ===== EDIT END =====

    } // Ends the loop through the five puzzle files.



    return 0; // Reports that the program completed successfully.

} // Ends the main function.
```

Correctness Analysis:

Both ChatGPT and Claude.ai had a hitch in the code that prevented them from functioning. When checking for and writing to files, the programs used "../" as a general directory as part of the filename search. This is mimicking what the previous student did, however the AIs did not work. Otherwise, the programs function as intended.

Time Complexity / Execution Time Analysis:

Comparing ChatGPT and Claude.ai, ChatGPT has the longer execution time, mainly due to its use of the vector library elements, and its use of extra variables in the main loop. These variables (puzzle Number and puzzles Processed) are for stopping the while loop and opening the correct files. Claude escapes this by using a for loop set for 5 iterations. This brings the variables count down.

Space Complexity Analysis:

Claude also has the superior space complexity due to its reduced library components and variables. Not having to store and retrieve the number of puzzles processed just barely cuts down on the space complexity.

Maintainability Analysis:

Claude.ai has a higher maintainability, mainly from its thorough file name checking and its terminal output that declares what file has been made for what file.

For the basis of this assignment Claude.ai will be chosen for rework. It was chosen for its superior execution time, space complexity, and maintainability.

Improvement Phase:

Claude.ai's code was improved in terms of correctness and maintainability. It was determined that the execution time and space complexity could not be easily improved. For correctness, the code was edited to fix the general directory issue. For maintainability, the comments were completely rewritten in a more thorough manner.